时序特征工程 · 知识点与代码速查手册

项目：time-series-feature-engineering ｜ 生成时间：2026-10-09 15:04 ｜ 打印建议：A4、缩放 100%（连续紧凑排版，已尽量减少空白）

> 配套项目：`time-series-feature-engineering`（北京 PM2.5 实战） 用途：把项目里用到的知识点和代码集中到一处，打印出来放在手边，写代码、写报告时对着查。 阅读方式：左边是代码，代码下面的「作用」就是这段代码在干什么，不需要背，知道去哪儿找就行。 重新生成 PDF：在项目根目录运行 `python guide\build_handbook.py`，PDF 会输出到 `outputs\知识点与代码速查.pdf`。 打印版 PDF：outputs/知识点与代码速查.pdf（已经在项目里生成好了，直接打开或打印即可）。

目录（连续排版，节省纸张）

时序特征工程 · 知识点与代码速查手册
0. 命令速查
1. Python 基础
2. pandas 基础
3. 时间戳处理
4. 时间特征
5. 滞后与滑动窗口
6. 建模与评估
7. 数据泄漏：三种错误示范
8. 画图
9. SQLite 数据库
10. 报错急救
附录 A 特征清单（可直接抄进报告）
附录 B 本项目的实验结论（真实数字）
附录 C 文件地图

怎么用这本手册

1. **先看「核心 10 句话」**：把整个项目的骨架记住，比记住代码重要。
2. **要写代码时，翻到对应章节**：每一节都是「代码 + 作用」，复制过去改成自己的列名就能用。
3. **要写报告时，看附录 A 和附录 B**：特征怎么解释、实验结论是什么，直接抄进报告。
4. **报错时看第 10 章**：按错误信息查，先看原因，再看解决办法。

核心 10 句话

1. 时序数据是带时间戳、按时间排好序的记录，**顺序本身就是信息**，不能随便打乱。
2. 时间戳统一用 **UTC**；北京时间 = UTC + 8。
3. 预测场景要先定义清楚：**站在 t 时刻，用截至 t 已知的信息，预测 t+1。**
4. 时间列可以拆成 **年、月、日、星期、小时、是否周末** 这些数字特征。
5. 小时、星期、月份用 **sin/cos 周期编码**，23 点和 0 点才会变成"邻居"。
6. **lag（滞后）** 是把过去的值挪到当前行：`shift(1)` 是上一小时，`shift(24)` 是昨天同一小时。
7. **rolling（滑动窗口）** 是算最近一段时间（如 24 小时）的均值、波动、最大最小值。
8. 任何用到了 t+1 及以后的信息，都是 **数据泄漏**，测试分数会假高。
9. 时序数据切分训练/测试必须 **按时间前后切**，不能随机切。
10. 先跑 **基线**（平均值、持续法），再谈复杂模型；指标看 **MAE、RMSE、R²**。

0. 命令速查

0.1 环境：创建并激活虚拟环境

```
# 在项目根目录（time-series-feature-engineering）执行
python -m venv .venv                          # 创建虚拟环境
.\.venv\Scripts\Activate.ps1                  # 激活（Windows PowerShell）
pip install -r requirements.txt               # 安装项目需要的库
```

作用：

- 虚拟环境 = 只属于这个项目的工具箱，不会污染电脑里其他 Python 项目。
- 激活成功后，命令行前面会出现 `(.venv)`；关掉窗口就相当于退出。
- Mac/Linux 用 `python3 -m venv venv` 和 `source venv/bin/activate`。

0.2 运行脚本

```
python scripts\05_time_features.py     # 运行单个脚本
.\run_all.ps1                          # 一键按顺序全部运行
```

作用：

- 脚本按编号顺序有依赖：04 生成干净数据 → 05 加时间特征 → 08 加滞后滑窗 → 09 做模型对比。
- 跳过前面的脚本直接跑 09，会提示找不到文件（`FileNotFoundError`）。

0.3 Git 与 GitHub

```
git init                                    # 把这个文件夹变成 Git 仓库
git add .                                   # 把改动放进"待提交"区
git commit -m "完成时间特征提取"             # 存一个版本，附上说明
git log --oneline                           # 看历史版本
git remote add origin https://github.com/你的用户名/仓库名.git
git push -u origin main                     # 第一次推送
git status                                  # 看当前有哪些改动（最常用）
```

作用：

- Git 是"存档 + 时间机器"，每完成一小步就 `add` + `commit` 一次。
- GitHub 是放代码的网站，导师要的是仓库链接，不是压缩包。
- `.gitignore` 用来排除不想上传的东西（本项目已排除 `.venv/`、`data/raw/` 等）。

0.4 网页指引、本地服务、生成手册

```
python guide\build_guide.py                                    # 重新生成网页指引
powershell -ExecutionPolicy Bypass -File .\guide\start-server.ps1   # 启动本地服务（能读文件夹）
python guide\build_handbook.py                                 # 重新生成这本手册的 PDF
```

作用：

- 网页指引：把教程、代码、图表、实验表格放在一个页面里看。
- 本地服务：浏览器出于安全不允许网页直接读本地文件夹，`start-server.ps1` 会开一个只服务本机的小服务器，网页里的"项目文件"才能读文件夹。
- 手册 PDF：改完上面的 Markdown 后跑一次，PDF 会重新生成。

---

1. Python 基础

对应脚本：`scripts/01_python_basics.py`；对应教程：`docs/02-Python零基础.md`。

1.1 列表 list：一串有序的数据

```
values = [80, 75, 90]        # 建列表
values.append(60)            # 末尾加一个
first = values[0]            # 取第 1 个（下标从 0 开始）
last = values[-1]            # 取最后一个
print(len(values))           # 长度：4
print(values[1:3])           # 切片：第 2、3 个
```

作用：存放一组有顺序的数据，比如一串连续的 PM2.5 观测值。

1.2 字典 dict：按名字取值

```
student = {"name": "小明", "score": 90}
print(student["name"])            # 取值：小明
student["score"] = 95             # 改值
student["city"] = "南京"           # 加新的一对
for key, value in student.items():  # 遍历
    print(key, value)
```

作用：用"名字 → 值"的方式存数据，读代码时比记 `[0]`、`[1]` 清楚得多。

1.3 队列 deque：先进先出

```
from collections import deque

window = deque(maxlen=3)      # 只保留最近 3 个
for value in [10, 20, 30, 40]:
    window.append(value)
print(window)                 # deque([20, 30, 40], maxlen=3)
print(window[0])              # 最早的那个：20
```

作用：维护"最近 N 个值"的滑动窗口；`maxlen` 会自动丢掉最旧的，这是滑动窗口最朴素的手写版本。

1.4 循环、条件、函数

```
def level(pm25: float) -> str:     # 函数：输入 → 输出
    if pm25 > 150:
        return "重度污染"
    elif pm25 > 75:
        return "轻度污染"
    return "正常"

for value in [30, 90, 200]:
    print(value, level(value))
```

作用：

- 函数把一段逻辑包起来，起个名字，后面重复用；`pm25: float` 和 `-> str` 只是提示类型，不写也能跑。
- `for` 用来遍历一串数据，`if/elif/else` 用来分支判断。

1.5 读文件、写文件

```
# 读：with 会在用完以后自动关文件，推荐写法
with open("notes/学习笔记模板.md", "r", encoding="utf-8") as handle:
    text = handle.read()
print(text[:200])                 # 只看前 200 个字符

# 写：w 表示覆盖写，a 表示在末尾追加
with open("outputs/notes.txt", "w", encoding="utf-8") as handle:
    handle.write("第一行\n第二行\n")
```

作用：

- 读写文件都要指定 `encoding="utf-8"`，否则中文在 Windows 上容易乱码。
- 读 CSV 这种表格数据时，用 pandas 更方便（见第 2 章）。

1.6 报错怎么读

```
# 报错最后一行才是"错误类型 + 原因"，先看它，再看它上面一行指出的是哪个文件哪一行
# 例：
#   File "scripts\05_time_features.py", line 42
#     df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
#   KeyError: 'hour'
```

作用：

- `KeyError: 'hour'` 意思是没有名为 `hour` 的列 —— 通常是你忘了先运行 04/05 脚本，或者列名拼错了。
- 详细对照表见第 10 章。

---

2. pandas 基础

对应脚本：`scripts/02_pandas_basics.py`；对应教程：`docs/03-pandas与SQLite.md`。

2.1 读 CSV

```
import pandas as pd

df = pd.read_csv("data/raw/beijing_pm25/PRSA_data_2010.1.1-2014.12.31.csv")
print(df.shape)        # (行数, 列数)
print(df.head())       # 看前 5 行
print(df.info())       # 每列的类型、缺失情况
```

作用：

- `pd.read_csv` 把 CSV 读成 DataFrame（可以理解成"程序里的 Excel 表格"）。
- 第一步永远先 `shape` / `head` / `info`：先看清数据长什么样，再动手。

2.2 选列、筛行、新增列

```
df["pm2.5"]                    # 选一列（Series）
df[["year", "month", "hour"]]  # 选多列（DataFrame）
df[df["pm2.5"] > 100]          # 筛出 PM2.5 大于 100 的行
df[(df["year"] == 2014) & (df["month"] == 1)]   # 多个条件用 & 连接，每个条件加括号

df["is_weekend"] = (df["dayofweek"] >= 5).astype(int)   # 新增一列
df["temp_f"] = df["TEMP"] * 9 / 5 + 32                  # 用已有列算新列
```

作用：

- 特征工程的大部分工作就是"新增列"：把原始信息加工成模型能用的数字。
- `.astype(int)` 把 True/False 变成 1/0，模型更喜欢数字。

2.3 排序、去重、缺失值

```
df = df.sort_values("datetime")          # 按时间排序
df = df.drop_duplicates(subset="datetime")   # 去掉重复时间戳

df.isna().sum()                          # 每列有多少缺失值
df["pm2.5"].isna().sum()                 # 目标列缺失多少
df = df.dropna(subset=["pm25_next"])     # 删掉"答案缺失"的行
df["pm2.5"] = df["pm2.5"].interpolate(method="linear", limit=6, limit_direction="both")
```

作用：

- 时序数据的第一原则：**先排序、再检查重复和缺失。**
- `interpolate(limit=6)` = 最多连续补 6 个小时；补不了的宁可留着，别硬猜（详细理由见 3.3）。

2.4 groupby：分组统计

```
df.groupby("hour")["pm2.5"].mean()               # 每个小时的平均 PM2.5
df.groupby("dayofweek")["pm2.5"].agg(["mean", "std", "count"])
df.groupby(["is_weekend", "hour"])["pm2.5"].mean()   # 多列组合分组
```

作用：

- 这是观察"周期性"最快的办法：如果不同小时的平均值差别明显，说明时间里有可用的信号。
- 结果通常直接画成柱状图（见第 8 章）。

2.5 时间列处理

```
df["datetime"] = pd.to_datetime(df[["year", "month", "day", "hour"]])  # 多列拼成一个时间
df["datetime"] = df["datetime"].dt.tz_localize("UTC")                  # 声明"这本来就是 UTC"
df = df.set_index("datetime").sort_index()                             # 时间当索引，再排序
df.index.hour          # 取小时；同类还有 .year .month .day .dayofweek .dayofyear
```

作用：

- 时间必须是真正的 datetime 类型，不能是字符串，否则没法做 `shift`、`rolling` 这些时间操作。
- 设成索引以后，`df.index.year` 就是每行的年份，写起来短。

2.6 保存 CSV

```
df.to_csv("data/processed/pm25_clean.csv", encoding="utf-8-sig")
```

作用：

- `encoding="utf-8-sig"` 让 Excel 打开中文表头不乱码，Windows 上推荐。
- `index=True`（默认）会把时间索引也写进去，这样别的脚本才能读回时间。

---

3. 时间戳处理

对应脚本：`scripts/04_load_clean.py`；对应教程：`docs/04-时序特征工程核心.md`。

3.1 拼出 UTC 时间列

```
import pandas as pd

dt = pd.to_datetime(df[["year", "month", "day", "hour"]])
df["datetime"] = dt.dt.tz_localize("UTC")
```

作用：

- 原始数据把时间拆成 4 列存，这一步把它们拼成一个带时区的时间列。
- `tz_localize` 的意思是"这些数字本来就是 UTC 时间"；如果原始时间是北京时间，就要先 `tz_localize("Asia/Shanghai")` 再 `tz_convert("UTC")`。
- 统一 UTC 是为了避免"两份数据一个用北京时间、一个用 UTC，合并后差 8 小时"。

3.2 排序 + 补齐完整时间轴

```
df = df.set_index("datetime").sort_index()
df = df.asfreq("h")          # 保证每个整点都有一行，缺的补成 NaN

expected = pd.date_range(df.index.min(), df.index.max(), freq="h")
print(len(expected), len(df), len(expected) - len(df))   # 应该有 / 实际有 / 缺多少
```

作用：

- `asfreq("h")` 让"每隔一小时一行"变成硬约束。
- 为什么必须补：如果不补，`shift(24)` 表示的是"往前数 24 行"，而不是"24 小时前"，遇到缺口就会错位，`rolling(24)` 也一样。

3.3 缺失值：短缺口插值，长缺口保留

```
df["pm25_was_missing"] = df["pm2.5"].isna().astype(int)   # 留个标记，方便以后分析
df["pm2.5"] = df["pm2.5"].interpolate(method="linear", limit=6, limit_direction="both")
```

作用：

- 连续缺失不超过 6 小时 → 用前后值线性插值补上（相邻小时污染水平接近，这样猜比较合理）。
- 连续缺失很久 → 保留 NaN，不硬猜；建模时按用到的特征删掉这些行。

- 插值也是"猜"，必须在报告里写清楚处理方式，不能装作数据本来没有缺失。

3.4 时区换算（要看北京作息时）

```
df["hour_beijing"] = (df["hour"] + 8) % 24                      # 简便算法
df["datetime_beijing"] = df.index.tz_convert("Asia/Shanghai")   # 正规做法
```

作用：

- 本项目统一用 UTC 讲解；UTC 08:00 = 北京 16:00。
- 想研究"北京早高峰"，就加上 `hour_beijing` 这一列再分组看。

---

4. 时间特征

对应脚本：`scripts/05_time_features.py`。

4.1 日历特征

```
index = df.index
df["year"] = index.year
df["month"] = index.month
df["day"] = index.day
df["hour"] = index.hour
df["dayofweek"] = index.dayofweek          # 0=周一, 6=周日
df["is_weekend"] = (index.dayofweek >= 5).astype(int)
df["dayofyear"] = index.dayofyear
```

作用：

- 把"时间戳"翻译成模型看得懂的数字：年看长期趋势，月看季节，小时看早晚高峰。
- `day`（几号）通常用处不大，容易让模型记住偶然现象（过拟合），可以先不加。
- `is_weekend` 是最直观的一个：工作日和周末的规律常常不一样。

4.2 星期几的可读版本

```
WEEKDAY_NAMES = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"]
df["weekday_name"] = df["dayofweek"].map(lambda x: WEEKDAY_NAMES[x])
```

作用：给人看的列，画图、写报告时比 0~6 直观；建模时用 `dayofweek` 数字列。

4.3 周期编码：sin/cos

```
import numpy as np

df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
df["hour_cos"] = np.cos(2 * np.pi * df["hour"] / 24)

df["dayofweek_sin"] = np.sin(2 * np.pi * df["dayofweek"] / 7)
df["dayofweek_cos"] = np.cos(2 * np.pi * df["dayofweek"] / 7)
```

作用：

- 直接给模型 `hour=23` 和 `hour=0`，模型会以为它们差 23；放到圆上以后，它俩其实是邻居。
- 除以周期长度（24 小时、7 天、12 个月），再用 sin/cos 算出圆上的横纵坐标，两个数配对表示一个位置。
- 月份编码写 `(month - 1) / 12`，是为了让 1 月从 0 开始，跟其他周期的写法一致。

4.4 观察周期性

```
by_hour = df.groupby("hour")["pm2.5"].mean()
by_weekday = df.groupby("weekday_name")["pm2.5"].mean().reindex(WEEKDAY_NAMES)
```

作用：分组求平均，看看"哪个小时更脏、哪一天更脏"。柱子高低差别明显，就说明时间特征有用。

5. 滞后与滑动窗口

对应脚本：`scripts/08_lag_rolling_features.py`。

5.1 lag：把过去的值挪到当前行

```
df["pm25_now"] = df["pm2.5"]        # 当前时刻的值（可以立刻测到，能用）
df["pm25_lag1"] = df["pm2.5"].shift(1)      # 上一小时
df["pm25_lag2"] = df["pm2.5"].shift(2)      # 上两小时
df["pm25_lag24"] = df["pm2.5"].shift(24)    # 昨天同一小时
df["pm25_lag168"] = df["pm2.5"].shift(168)  # 上周同一小时（7×24）
```

作用：

- `shift(1)` 把整列往下挪一格，于是"上一小时的值"出现在当前行，可以当作特征。
- 污染物有明显惯性，`lag1`、`lag24` 通常是最有用的特征。
- 挪完前几行会出现 NaN，这是正常的，最后 `dropna` 掉即可。

5.2 diff：变化量

```
df["pm25_diff1"] = df["pm2.5"].diff(1)     # 比上一小时高/低了多少
df["pm25_diff24"] = df["pm2.5"].diff(24)   # 比昨天同一小时高/低了多少
```

作用：`diff` 相当于"当前值 − 之前的值"，描述的是趋势和变化速度，对突变更敏感。

5.3 rolling：滑动窗口

```
window24 = df["pm2.5"].rolling(window=24, min_periods=18)

df["pm25_roll24_mean"] = window24.mean()   # 最近 24 小时平均
df["pm25_roll24_std"] = window24.std()     # 最近 24 小时波动
df["pm25_roll24_max"] = window24.max()     # 最近 24 小时最高
df["pm25_roll24_min"] = window24.min()     # 最近 24 小时最低

df["pm25_roll168_mean"] = df["pm2.5"].rolling(window=168, min_periods=120).mean()  # 最近 7 天
```

作用：

- 默认窗口在当前时刻结束（trailing），里面全是过去的值，预测下一小时时是安全的。
- `min_periods=18` 表示"24 小时里至少有 18 个有效值才算"，既不会被个别缺失整行丢掉，也不会只靠一两个值硬算平均。
- `mean` 看水平、`std` 看稳定程度、`max/min` 看极端情况。

5.4 目标：下一个小时的值

```
df["pm25_next"] = df["pm2.5"].shift(-1)    # 向上挪一格 = 下一小时的值
df = df.dropna(subset=["pm25_next"])       # 最后一行没有下一小时，删掉
```

作用：

- 这一列是**答案（y）**，只能用来训练和打分，绝不能出现在特征列里。
- `shift(-1)` 和 `shift(1)` 方向相反：负数 = 未来，正数 = 过去。

5.5 时间对齐：一张表看懂谁对应谁

列名	对应时刻	说明
`pm2.5`	t	当前小时测得的值
`pm25_lag1`	t-1	上一小时
`pm25_lag24`	t-24	昨天同一小时
`pm25_roll24_mean`	t-23 ~ t	最近 24 小时的平均

列名	对应时刻	说明
`pm25_next`	t+1	要预测的答案

作用：写代码时随时回看这张表，就不容易把未来信息混进特征（这是最常见的错误）。

---

6. 建模与评估

对应脚本：`scripts/06_baseline_model.py`、`07_cyclic_encoding.py`、`09_model_comparison.py`。

6.1 准备好 X 和 y

```
features = ["month", "day", "hour", "dayofweek", "is_weekend", "dayofyear"]

x_train = train[features]      # 特征（问题）
y_train = train["pm25_next"]   # 目标（答案）
```

作用：

- 建模就是把"能知道的信息"（X）和"要预测的答案"（y）分清楚。
- 特征必须满足一个条件：**在预测时刻已经能拿到。**

6.2 按时间切分训练集/测试集

```
split_index = int(len(df) * 0.8)
train = df.iloc[:split_index].copy()     # 前 80%：训练
test = df.iloc[split_index:].copy()      # 后 20%：测试
```

作用：

- 时序数据必须"用过去训练、拿未来测试"，模拟真实预测场景。
- 随机切分会让未来信息混进训练集，分数虚高（第 7 章有真实数字对比）。

6.3 基线模型：先跑简单的

```
from sklearn.dummy import DummyRegressor

dummy = DummyRegressor(strategy="mean")   # 永远预测训练集的平均值
dummy.fit(x_train, y_train)
dummy_pred = dummy.predict(x_test)

persistence_pred = test["pm25_now"].to_numpy()   # 持续法：下一小时 = 当前小时
```

作用：

- 平均值基线：什么都不学，只猜平均值，用来当"最低标准"。
- 持续法：直接说"下一小时和现在差不多"。在污染物、股价这类有惯性的数据上，它往往很强。
- 复杂模型如果打不过这两个基线，说明特征或方法有问题。

6.4 线性回归

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(x_train, y_train)      # 训练
pred = model.predict(x_test)     # 预测

for name, coef in zip(features, model.coef_):
    print(name, round(coef, 4))  # 看每个特征的系数
```

作用：

- 线性回归就是给每个特征学一个权重，然后加权求和。

- 系数正负表示方向（正 = 越大越容易高），但系数大小受量纲影响，不能直接当重要性排名；想比较大小要先标准化。

6.5 标准化后再看系数

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(train[features])
x_test_scaled = scaler.transform(test[features])     # 注意：用训练集的规则去转换测试集

model = LinearRegression().fit(x_train_scaled, y_train)
```

作用：把所有特征缩放到可比的尺度，系数大小才能横向比较（本项目输出到 `09_feature_coefficients.csv`）。

6.6 三个指标怎么读

```
mae = np.mean(np.abs(y_true - y_pred))                    # 平均差多少
rmse = np.sqrt(np.mean((y_true - y_pred) ** 2))           # 更重罚大错
r2 = 1 - np.sum((y_true - y_pred) ** 2) / np.sum((y_true - y_true.mean()) ** 2)
```

作用：

- **MAE**：平均每次预测差多少，最好和 PM2.5 本身的平均值一起看（本数据平均约 80~100，MAE 11 就不错）。
- **RMSE**：和 MAE 类似，但对离谱的大错误更敏感。
- **R²**：1 最好；0 约等于"只会猜平均值"；负数说明比猜平均值还差。

6.7 实验怎么写

```
实验名 | 用了哪些特征 | 切分方式 | MAE | RMSE | R² | 是否可信
```

作用：

- 一次只改一处，其他保持不变，这样的对比才有意义（控制变量）。
- 表里一定要标出"是否可信"，把泄漏实验单独区分开（项目里已有这张表：`outputs/tables/09_model_comparison.csv`）。

---

7. 数据泄漏：三种错误示范

对应脚本：`scripts/08_lag_rolling_features.py`、`scripts/09_model_comparison.py`。

7.1 中心滑动窗口（看了未来）

```
# 错误：center=True 表示窗口中心在当前行，会包含 t+1、t+2 的未来值
df["pm25_roll24_mean_center_leaky"] = df["pm2.5"].rolling(window=24, center=True, min_periods=24).mean()
```

作用/后果：预测下一小时时却用了后面几小时的信息。分数会变好，但这个"好"是假的。

7.2 把答案当特征（最离谱的作弊）

```
df["pm25_answer_leaky"] = df["pm25_next"]     # 严重错误：答案直接变成特征
```

作用/后果：模型等于抄答案，R² 会到 1.0000、MAE 会到 0。看到"完美分数"先怀疑泄漏，而不是高兴。

7.3 随机切分（考试漏题）

```
# 错误做法：时序数据不要这样切
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

作用/后果：随机切分会把"未来"混进训练集，测试分数虚高（本项目对比：随机切分 R² 0.9494，按时间切分 0.9426，看起来更好，但不可信）。

7.4 防泄漏检查清单

- [ ] 每一列特征，在预测时刻 t 都拿得到吗？（拿不到就删）
- [ ] 有没有 `center=True`？有就改成默认的 trailing 窗口。
- [ ] 有没有列名带 `next`、`answer`、`leaky` 却出现在 `features` 里？
- [ ] 是在排序 + 补齐时间轴之后才算的 `shift`/`rolling` 吗？
- [ ] 训练集时间是否完全早于测试集？
- [ ] 分数好得反常（R^2>0.99）时，是否停下来查一遍泄漏？

---

8. 画图

对应脚本：`scripts/10_plots.py`；对应教程：`docs/06-画图与Kaggle.md`。

8.1 中文画图设置

```
import matplotlib
matplotlib.use("Agg")            # 不弹窗，直接存图片文件

import matplotlib.pyplot as plt
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei"]   # 中文不乱码
plt.rcParams["axes.unicode_minus"] = False                        # 负号正常显示
plt.rcParams["figure.dpi"] = 120
```

作用：不设置字体，中文标题会变成一排方框；`Agg` 让脚本在服务器/无窗口环境也能出图。

8.2 常用图

```
fig, axes = plt.subplots(2, 1, figsize=(12, 8))    # 上下两张图

axes[0].plot(daily.index, daily["pm2.5"], color="#2c7fb8")        # 折线：看趋势
axes[0].set_title("PM2.5 日均值（UTC）")
axes[0].set_ylabel("PM2.5")

axes[1].plot(sample.index, sample["pm2.5"], linewidth=0.8)        # 局部放大看细节
fig.tight_layout()
fig.savefig("outputs/figures/01_原始序列与趋势.png", bbox_inches="tight")
```

```
plt.figure(figsize=(8, 4))
plt.bar(by_hour.index, by_hour.values, color="#31a354")           # 柱状：比较不同小时
plt.xlabel("UTC 小时"); plt.ylabel("平均 PM2.5")
plt.savefig("outputs/figures/03_周期性观察.png", bbox_inches="tight")
```

```
fig, ax = plt.subplots(figsize=(13, 5.5))
ax.plot(part.index, part["y_true"], color="black", linewidth=2, label="真实值")
ax.plot(part.index, part["pred_time_cyc"], color="#e6550d", label="只用周期时间特征")
ax.plot(part.index, part["pred_full"], color="#2c7fb8", label="完整正确特征")
ax.legend()
ax.set_title("测试期前 14 天：预测值 vs 真实值")
```

作用：

- 折线看趋势和周期，柱状比较分组均值，散点看两个变量的关系。
- 预测对比图是最有说服力的一张：真实值、简单特征、完整特征画在一起，一眼看出差别。

8.3 每张图在说什么

图	看什么
原始序列	整体趋势、有没有季节性
滑动平均与差分	7 天滑动平均像降噪滤镜；差分把突变单独拎出来
周期性观察	不同小时/星期/月份的平均值差别大不大

图	看什么
预测对比	简单特征几乎是一条平线，完整特征能跟住真实值
周期编码圆	23 点和 0 点在圆上是邻居
相关性热力图	哪个特征和下一小时 PM2.5 线性相关更强（相关 ≠ 因果）

9. SQLite 数据库

对应脚本：`scripts/03_sqlite_basics.py`。Python 自带，零配置。

9.1 连接与建表

```
import sqlite3

conn = sqlite3.connect("data/processed/practice.db")
conn.execute("""
    CREATE TABLE IF NOT EXISTS weather_notes (
        id    INTEGER PRIMARY KEY,   -- 主键：唯一身份证，不能重复
        city  TEXT NOT NULL,
        date  TEXT NOT NULL,
        temp  REAL,
        note  TEXT
    )
""")
conn.commit()
```

作用：

- SQLite 把整个数据库存成一个 `.db` 文件，不用装服务器，适合练习和小项目。
- `PRIMARY KEY` 保证每行有唯一 id；`NOT NULL` 表示这列不能空着。
- 类型：`INTEGER` 整数、`TEXT` 文本、`REAL` 小数。

9.2 增删改查

```
# 增：? 是占位符，防止拼字符串出错（也能防注入）
conn.executemany(
    "INSERT OR REPLACE INTO weather_notes (id, city, date, temp, note) VALUES (?, ?, ?, ?, ?)",
    [(1, "Nanjing", "2026-10-01", 28.0, "sunny")],
)

# 查
conn.execute("SELECT * FROM weather_notes WHERE city = ? ORDER BY id", ("Nanjing",))
conn.execute("SELECT city, COUNT(*) AS n FROM weather_notes GROUP BY city ORDER BY n DESC")

# 改
conn.execute("UPDATE weather_notes SET temp = ?, note = ? WHERE id = ?", (29.5, "updated", 1))

# 删
conn.execute("DELETE FROM weather_notes WHERE id = ?", (4,))

conn.commit()    # 增删改之后一定要 commit
conn.close()     # 用完关掉
```

作用：

- `SELECT` 查、`INSERT` 增、`UPDATE` 改、`DELETE` 删 —— 数据库的四个基本动作。
- `WHERE` 决定动哪几行；**忘了写 WHERE，UPDATE/DELETE 会作用到整张表**，务必先 `SELECT` 确认。

9.3 主键与索引

```
conn.execute("CREATE INDEX IF NOT EXISTS idx_weather_city ON weather_notes (city)")
```

作用：

- 主键：自动唯一，插入重复 id 会直接报 `IntegrityError`（这是好事，数据更安全）。
- 索引：像书的目录，让"按 city 查询"更快；代价是占空间、写入稍慢，只给常用查询列建。

---

10. 报错急救

对应教程：`docs/99-报错急救.md`。

报错信息（关键部分）	原因	怎么解决
`python 不是内部或外部命令`	没装 Python 或没勾选 Add to PATH	重新安装 Python 并勾选 Add Python to PATH；或用 `py` 命令
`无法加载文件 ... Activate.ps1，未对文件进行数字签名`	PowerShell 执行策略限制	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` 后重开窗口
`ModuleNotFoundError: No module named 'pandas'`	没装库，或虚拟环境没激活	先 `.\.venv\Scripts\Activate.ps1`，再 `pip install -r requirements.txt`
`FileNotFoundError: ... data/raw/...csv`	数据还没下载	先跑 `python scripts\download_data.py`
`FileNotFoundError: ... pm25_clean.csv`	上一个脚本没跑	按顺序跑 04 → 05 → 06...，或直接 `.\run_all.ps1`
`KeyError: 'hour'`	列名不存在/拼错，或少了上一步生成的特征	用 `df.columns` 看真实列名；确认跑过 05/08
`ValueError: could not convert string to float`	该是数字的列里混了文字/空值	`df["列"].head()` 看看内容；必要时 `pd.to_numeric(errors="coerce")`
`TypeError: unsupported operand type(s)`	字符串当数字算了	检查这列的 `dtype`（`df.dtypes`），先转换类型
`SettingWithCopyWarning`	在切片副本上改数据	先 `.copy()`，再赋值：`part = df.iloc[:100].copy()`
`UnicodeDecodeError: 'utf-8' codec can't decode`	文件不是 UTF-8（常见 GBK）	读的时候加 `encoding="gbk"`；写 CSV 用 `utf-8-sig`
CSV 用 Excel 打开中文乱码	编码不是 Excel 认的格式	保存时用 `encoding="utf-8-sig"`
matplotlib 中文变方框	没设置中文字体	加 `plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"]`
服务器提示端口被占用	8765 已被别的程序占用	换端口：`start-server.ps1 -Port 8899`
Git `fatal: not a git repository`	当前目录还不是仓库	在项目根目录先 `git init`
Git `failed to push` / 要登录	远端仓库或凭据问题	`git remote -v` 检查地址；按提示登录 GitHub
`PermissionError: [Errno 13]`	文件被 Excel 等程序占用	关掉正在打开的文件再运行脚本

通用排查三步：

1. 看报错**最后一行**（错误类型 + 原因），再看它指出的是哪个文件哪一行。
2. 打开那个文件、找到那一行，想想这一行需要什么（列？文件？库？）。
3. 还没头绪就把整段报错复制去搜索，或者把"最后一行 + 代码那几行"发给同学/导师。

附录 A　特征清单（可直接抄进报告）

特征名	构造方法	含义	可能的作用
`year`	`index.year`	年份	捕捉长期趋势
`month`	`index.month`	月份 1–12	捕捉季节变化

特征名	构造方法	含义	可能的作用
`day`	`index.day`	几号	通常用处不大，容易过拟合
`hour`	`index.hour`	小时 0–23	捕捉一天内的变化
`dayofweek`	`index.dayofweek`	0=周一…6=周日	工作日/周末差异
`is_weekend`	`dayofweek >= 5`	是否周末 0/1	简单明了的周末效应
`dayofyear`	`index.dayofyear`	一年中的第几天	更细的季节位置
`hour_sin` / `hour_cos`	`sin(2π·hour/24)` / `cos(...)`	小时在圆上的坐标	让 23 点和 0 点变近
`dayofweek_sin` / `dayofweek_cos`	`sin(2π·dow/7)` / `cos(...)`	星期在圆上的坐标	表达"周而复始"
`month_sin` / `month_cos`	`sin(2π·(month−1)/12)`	月份在圆上的坐标	表达季节循环
`pm25_now`	原列	当前小时的值	惯性，最强特征之一
`pm25_lag1 / lag2 / lag3`	`shift(1/2/3)`	前 1~3 小时的值	近期惯性
`pm25_lag24`	`shift(24)`	昨天同一小时	日周期
`pm25_lag168`	`shift(168)`	上周同一小时	周周期
`pm25_diff1 / diff24`	`diff(1) / diff(24)`	相比上一小时/昨天同小时的变化	趋势与突变
`pm25_roll24_mean/std/max/min`	`rolling(24).mean()/std()/max()/min()`	最近 24 小时统计	水平、波动、极值
`pm25_roll168_mean/std`	`rolling(168)`	最近 7 天统计	周级别水平与波动
`TEMP_lag1` / `TEMP_roll24_mean`	对温度做同样的操作	温度的历史信息	说明方法可迁移到其他列

一句话模板：`xxx 特征是用 xxx 方法构造的，它让模型知道 xxx，加入后 MAE 从 __ 降到 __。`

附录 B　本项目的实验结论（真实数字）

B.1 特征对照实验（`outputs/tables/09_model_comparison.csv`）

实验	特征	切分方式	MAE	RMSE	R^2	是否可信
平均值基线	无	按时间 8:2	64.740	82.470	-0.0286	可信
持续法基线	当前 pm2.5	按时间 8:2	10.981	19.968	0.9397	可信
只用原始时间特征	month/day/hour/weekday…	按时间 8:2	62.576	81.323	-0.0002	可信
只用周期编码时间	hour_sin/cos…	按时间 8:2	62.656	80.601	0.0175	可信
周期编码 + 滞后	时间 + lag/diff	按时间 8:2	10.934	19.670	0.9415	可信
完整正确版	周期 + 滞后 + 滑动 + 天气	按时间 8:2	10.754	19.473	0.9426	可信
中心滑动窗口（泄漏）	窗口包含未来值	按时间 8:2	10.413	18.112	0.9504	不可信
把答案当特征（作弊）	`pm25_answer_leaky`	按时间 8:2	0.000	0.000	1.0000	不可信
随机切分（错误做法）	完整正确特征	随机 8:2	12.430	21.106	0.9494	不可信

B.2 消融实验（`outputs/tables/09_ablation.csv`）

实验	特征个数	MAE	RMSE	R^2
完整特征	29	10.754	19.473	0.9426
去掉滞后	21	33.376	45.330	0.6892
去掉滑动窗口	23	10.804	19.511	0.9424
去掉天气	21	10.885	19.634	0.9417
只留时间特征	7	62.656	80.601	0.0175

B.3 三条可以写进报告的结论

1. **只用钟表时间不够**：只加月/日/小时/星期，MAE 62.6，和"猜平均值"（64.7）差不多；换成周期编码也只有极小改善。
2. **过去的值才是主力**：加上滞后和滑动窗口后，MAE 从 62.6 降到 10.9；消融实验里"去掉滞后"这一项 MAE 直接涨到 33.4，说明这类特征贡献最大。
3. **分数越高越要警惕**：中心滑动窗口（10.413）和"把答案当特征"（MAE 0、R^2 1.0）分数更漂亮，但都是泄漏，不能写进结论。

附录 C　文件地图

位置	里面是什么
`README.md`	项目总览、7 天学习计划、导师要求对照
`docs/00` ~ `docs/07`	分步教程（导师要求翻译、环境、Python、pandas/SQLite、特征工程核心、建模、画图与 Kaggle、交作业模板）
`docs/08-知识点与代码速查.md`	**本手册的 Markdown 版本**
`docs/99-报错急救.md`	报错对照表（比本手册第 10 章更详细）
`scripts/01` ~ `03`	Python、pandas、SQLite 基础练习
`scripts/04` ~ `08`	数据清洗 → 时间特征 → 基线模型 → 周期编码 → 滞后滑窗
`scripts/09`	模型对比、消融实验、泄漏演示
`scripts/10`	画 6 张观察图
`scripts/common.py`	公共函数：路径、读数据、指标、中文字体
`data/raw/`	原始数据（下载得到）
`data/processed/`	清洗后数据、特征数据、练习数据库
`outputs/figures/`	6 张图（PNG，可插入报告）
`outputs/tables/`	实验表格（CSV，可插入报告）
`outputs/知识点与代码速查.pdf`	**本手册的 PDF（打印用）**
`guide/`	网页版指引：`index.html`、本地服务器 `server.py`、生成脚本

附录 D　交作业自查

- [ ] 代码能一键跑通：`.\run_all.ps1`
- [ ] 代码已推到 GitHub，仓库链接能打开
- [ ] 笔记里有每个特征的"构造 + 含义 + 实验证据"（用 `notes/学习笔记模板.md`）
- [ ] 实验记录表填好：模型对比、消融实验（用 `notes/实验记录模板.md`）
- [ ] 报告里有 1~2 张图（建议：原始序列+滑动平均、预测对比）
- [ ] 报告里写清楚了：预测目标是什么、怎么切分、有没有数据泄漏、结论是什么
- [ ] 能口头解释：为什么 23 点和 0 点是邻居、为什么要 shift、为什么不能随机切分